

ARQUITECTURA DE SOFTWARE

PRÁCTICO



```
{  
  "nombre": "Tomás Cassanelli",  
  
  "edad": 24,  
  
  "titulo": "Ingeniero en Sistemas",  
  
  "rol": "Profesor",  
  
  "materias": [  
    "Desarrollo de Software", "Arquitectura de Software"  
  ],  
  
  "sobre_mi": {  
    "estilo": "Clases prácticas",  
  
    "forma": "Confianza + compromiso = buenos resultados",  
  
  }  
}
```

Arq. Software. - Practico Tomi

Grupo de WhatsApp



TomiCassanelli/ **arquitectura-de-software**

Materia de Ingenieria en Informatica - UCC



1

Contributor

0

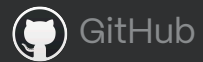
Issues

0

Stars

0

Forks



GitHub – TomiCassanelli/arquitectura-de-software: Materia de ...

Materia de Ingenieria en Informatica – UCC. Contribute to TomiCassanelli/arquitectura-de-software development by creating an...

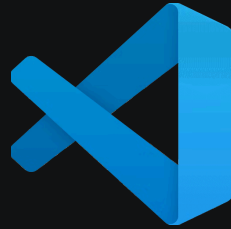
Stack Tecnológico

Vamos a utilizar un conjunto de herramientas durante todo el curso. Estas tecnologías son las mismas que utilizan empresas reales en proyectos reales. NECESITAMOS TENERLAS A TODAS!



Git

Sistema de control de versiones para coordinar el trabajo en grupo.



Visual Studio Code

Editor de código con extensiones que facilitan el desarrollo web.



Golang

Lenguaje open source diseñado por Google para ser simple, eficiente y rápido.



Chrome / Brave

Navegadores web con herramientas de desarrollo integradas.



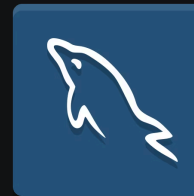
Node.js

Necesario para armar el entorno de trabajo del frontend, poder tener acceso a npm que es un gestor de paquetes, typescript y poder levantar ese entorno.



Postman / Swagger

Herramienta esencial para probar y documentar APIs, permitiéndonos enviar requests y ver respuestas de forma visual.



SQL Workbench

Interfaz gráfica para gestionar bases de datos, visualizar tablas y ejecutar consultas SQL de manera eficiente.



Docker

Permite crear entornos de desarrollo aislados y reproducibles, asegurando que tu aplicación funcione igual en cualquier máquina.

ARQUITECTURAS DE SOFTWARE

De Monolitos a Microservicios

PARTE 1 · TEORÍA

¿Qué es la Arquitectura de Software?

La arquitectura define cómo se organiza un sistema: su calidad, mantenibilidad y escalabilidad dependen de ella. A medida que los proyectos crecen, suele aparecer la transición hacia modelos distribuidos.

Arquitectura Monolítica

Toda la lógica (UI, negocio y datos) empaquetada en un único despliegue.

Single-process Monolith

Todo el código en un solo proceso ejecutable.

Monolito Modular

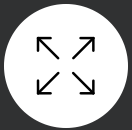
Módulos lógicos, pero contruidos y desplegados juntos.

Monolito Distribuido

Anti-patrón: servicios separados con alto acoplamiento que obligan a desplegar todo junto.

Arquitectura de Microservicios

Servicios pequeños, independientes y poco acoplados. Cada uno tiene su propia base de datos y se despliega de forma autónoma.



Escalabilidad Independiente

Cada servicio escala según su demanda.



+ Mantenibilidad

Actualizar partes pequeñas es más manejable.



+ Flexibilidad

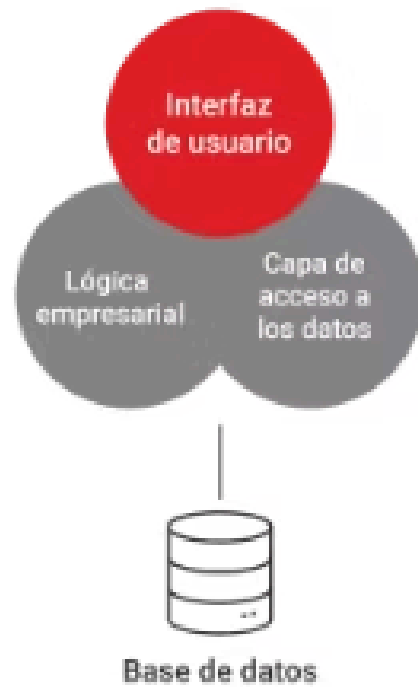
Distintos lenguajes según el problema.



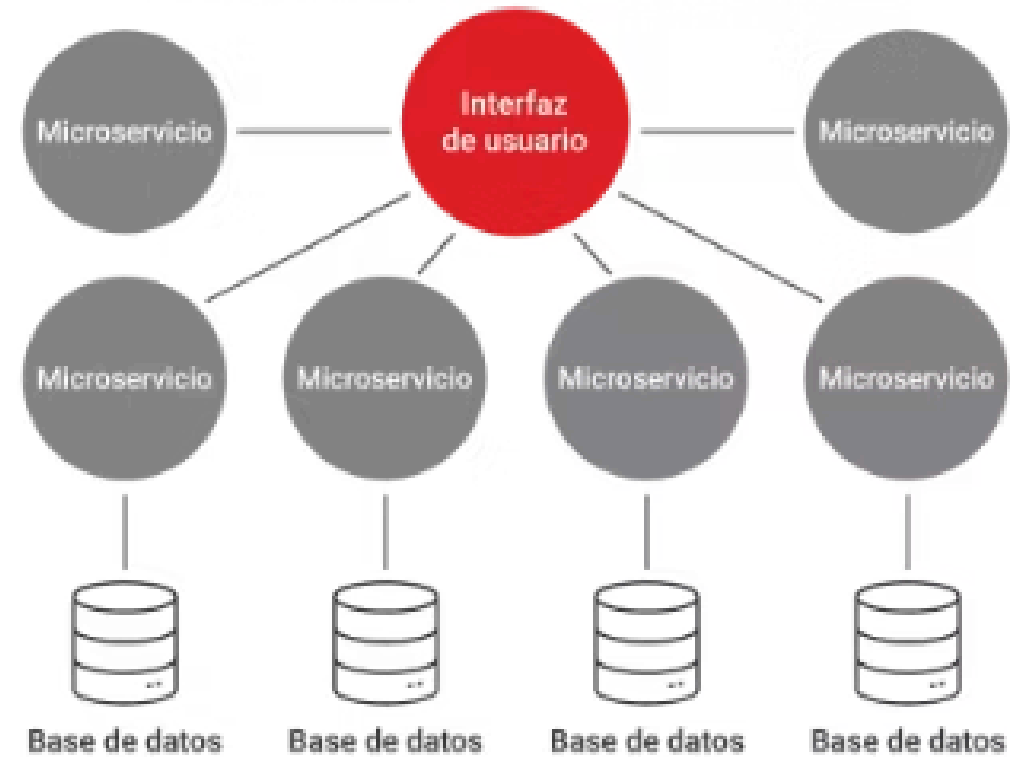
Paralelización

Equipos autónomos trabajan sin interferencias.

ARQUITECTURA MONOLÍTICA

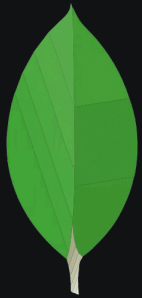


ARQUITECTURA DE MICROSERVICIOS



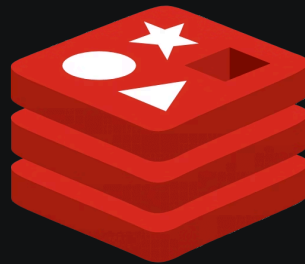
Se suman algunas más al Stack Tecnológico

Vamos a utilizar un conjunto de herramientas durante todo el curso. Estas tecnologías son las mismas que utilizan empresas reales en proyectos reales. NECESITAMOS TENERLAS A TODAS!



MongoDB

Como base de datos NoSQL orientada a documentos para manejar datos no estructurados.



MemCached / Redis

Como herramientas de caché, que guardan datos temporalmente.



Solr

Como plataforma de búsqueda. Nos permite indexar datos y realizar búsquedas sofisticadas.



RabbitMQ

Como sistema de mensajería que permite comunicación asíncrona.

Definición de Endpoints RESTful

RESTful define reglas para nombrar recursos en APIs. En Go (con Gin o router estándar), la convención es:

 **Regla de oro:** `router.MÉTODO("/entidad_en_plural/:id", Controlador.Acción)`

Componentes de la URI

- **Método HTTP:** GET, POST, PUT, PATCH, DELETE
- **Entidad:** nombre en plural (ej. users, albums, properties)

Problema

Imaginen que armaron el sistema para un gimnasio chico. Era hermoso por donde lo veas, ordenado en Go, con una sola base de datos relacional que manejaba todo, y lo vendieron caro. Pero el gimnasio, en parte gracias a la excelente organización de su sistema, se convirtió en un éxito absoluto. Abrieron cinco sucursales (haciéndole competencia directa a Qivox), sumaron clases de Hyrox, funcional y yoga con cupos estrictos. Además, agregaron una tienda online para vender indumentaria y suplementos, armaron un espacio con café de especialidad, se integraron con apps externas y lanzaron un club de beneficios. Hoy en día, el sistema actual ya no da abasto y los despliegues y el mantenimiento tienen riesgos cada vez más grandes.

Consigna: En base a la teoría, debatan en grupo y listen al menos tres problemas críticos o cuellos de botella técnicos que el equipo de desarrollo está sufriendo hoy en día con este monolito.

Primera Decisión

Escenario: Tras los problemas detectados, la decisión técnica es migrar hacia una arquitectura de microservicios. El objetivo es lograr un bajo acoplamiento y alta cohesión.

- **Consigna:** Sin escribir código ni pensar en bases de datos, ¿en cuántos microservicios dividirían el nuevo ecosistema del gimnasio? Nombren cada microservicio y escriban una breve oración indicando cuál es su responsabilidad principal (¡No se olviden de la tienda y el café!).

A cranearlo

- **Consigna:** Respetando las buenas prácticas de diseño RESTful (entidades uniformes en plural, uso correcto de verbos HTTP), escriban las líneas del router en Go definiendo **4 endpoints para cada uno** de los siguientes dominios:
 - a. Dominio de Usuarios (Identidad).
 - b. Dominio de Actividades (Clases/Hyrox).
 - c. Dominio de E-Commerce (Tienda/Café).
 - d. Dominio de Beneficios (Loyalty).

Microservicios con Interfaces y Mocks

Imaginemos que estamos programando el Microservicio de Actividades. Alguien quiere reservar su clase de Hyrox. Por regla de negocio, antes de darle el cupo, tenemos que preguntarle al Microservicio de Usuarios si esa persona tiene la cuota al día.

Como ahora son microservicios separados, la única forma de comunicarse es haciendo una petición HTTP (un GET) por la red.

¿Cuál es el problema acá?

Si el equipo que programa 'Usuarios' todavía no terminó su código, o si no tenemos internet, nuestro microservicio de 'Actividades' se queda trabado esperando. Dependemos 100% de que el otro exista y funcione para poder probar nuestro propio código. Es un semáforo en rojo para nuestro desarrollo.

La Solución

Para solucionar esto, en la arquitectura de software no dependemos de implementaciones reales, dependemos de Contratos (Interfaces).

Imaginen que nuestro controlador de Actividades es un televisor. El televisor no sabe cómo se genera la electricidad, solo sabe que si se conecta a un enchufe estándar de dos patas (el Contrato), va a funcionar.

Gracias a ese enchufe estándar, podemos conectarlo a la pared (la petición HTTP real) o podemos conectarlo a un generador a batería falso (el Mock). Al televisor no le importa, mientras le den energía.

Demostración en rama "clase-1": [Click Aqui](#)

Actividad para la Semana que Viene 🤪

Como vimos en clase, nuestro sistema del gimnasio arrancó siendo un monolito hermoso en Go... La cuestión es que si se rompe el módulo del café, ¡la gente no puede entrar a entrenar!

Desde el repo actual, en la rama de la clase, se presenta el TP

Como verán, es un monolito absoluto: todos los controladores conviven en el mismo archivo y todas las rutas se levantan en un único servidor escuchando en el puerto 8080.

Actividad: Deberán reescribir este código dividiéndolo en **cuatro microservicios independientes y además, aplicar interfaces y mocks para al menos uno.**

1. Microservicio de Usuarios
2. Microservicio de Actividades
3. Microservicio de Tienda (E-Commerce/Café)
4. Microservicio de Beneficios (Loyalty)